```
$dom = Dom\HTMLDocument::createFromString($html5);
$xpath = new Dom\XPath($dom);
$xpath->registerNamespace('ns', $dom->documentElement->namespaceURI);
$xpath->registerNamespace('php', 'http://php.net/xpath');
```

With all that set up, you can now start registering your own PHP functions for use in an XPath query. Here we'll pass a `callable` as an anonymous function, for extra cool points (and avoid polluting global scope with an extra function name):

```
$xpath->registerPhpFunctions(['containsLovely' => function
(array|string $arg) {
    print "calling containsLovely() ..." . PHP_EOL;
    if (is_string($arg)) {
        print "testing a string ..." . PHP_EOL;
        return str_contains($arg, 'lovely');
    }

    foreach ($arg as $node) {
        if ($node->nodeType == XML_ELEMENT_NODE) {
            print "testing a Dom\Element ..." . PHP_EOL;
            return str_contains($node->textContent, 'lovely');
        }
    }

    return false;
}]);
```

This function doesn't do anything particularly amazing; at heart, it's just a `str_contains()` call with some parameter handling. We could have just registered `str_contains()` like this: `Dom\XPath::registerPhpFunctions('str_contains');` but then we'd need to put extra logic into the XPath expression itself. This way we're keeping our expression concise (always a good idea) as well as creating a learning opportunity, which you'll see in a moment. To call the function mapped to the `containsLovely` string, we use the syntax `php:function(<NAME>, <ARG>)` where `<ARG>` is an XPath localization step:

```
$result = $xpath->query(
  '//ns:head[php:function("containsLovely", ns:title)]'
);
print "Found {$result->length} matching node(s)" . PHP_EOL;
```